Finite Automata Simulator

Raymond Claudio, Jacob Gainley, Aron Littlejohn, Cameron Long, and Wyatt Rose

CPSC 362 : Computing Theory

Embry-Riddle Aeronautical University

Instructor Ravi Ramnarayan

August 23, 2026

Finite Automata Simulator

Introduction

This C++ program allows a user to define and simulate either a deterministic finite automaton (DFA) or nondeterministic finite automaton (NFA). The user provides the states, alphabet, start state, accepting states, and transition table. The simulator then processes input strings and determines whether each string is accepted or rejected.

Deterministic Finite Automaton

A DFA has exactly one destination state for each state and alphabet-symbol combination. The simulator follows one transition path and reports its final state.

Nondeterministic Finite Automaton

An NFA can have zero, one, or multiple destination states for a state and symbol combination. The simulator tracks every active state and accepts the string when at least one final state is an accepting state.

Purpose

This test plan verifies DFA and NFA creation, state-transition processing, string acceptance, input validation, empty-string handling, and normal program termination.

DFA Test Configuration

The DFA accepts binary strings ending in 1.

Component	Definition
States	q0, q1
Alphabet	0, 1
Start state	q0
Accepting state	q1
q0 on 0	q0
q0 on 1	q1
q1 on 0	q0
q1 on 1	q1

Planned DFA Tests

Test ID	Input	Expected Result
DFA-01	1	Accepted
DFA-02	101	Accepted
DFA-03	100	Not Accepted

Test ID	Input	Expected Result
DFA-04	EMPTY	Not Accepted
DFA-05	102	Invalid symbol is reported
DFA-06	QUIT	Simulator closes normally

NFA Test Configuration

The NFA accepts binary strings containing at least one 1.

Component	Definition
States	q0, q1
Alphabet	0, 1
Start state	q0
Accepting state	q1
q0 on 0	q0
q0 on 1	q0 and q1
q1 on 0	q1

Component	Definition
q1 on 1	q1

Planned NFA Tests

Test ID	Input	Expected Result
NFA-01	000	Not Accepted
NFA-02	001	Accepted
NFA-03	1010	Accepted
NFA-04	EMPTY	Not Accepted
NFA-05	12	Invalid symbol is reported
NFA-06	QUIT	Simulator closes normally

Validation Tests

Test ID	Invalid Entry	Expected Behavior
VAL-01	Automata type other than DFA or NFA	Entry is rejected and requested again
VAL-02	Duplicate state name	Duplicate is rejected

Test ID	Invalid Entry	Expected Behavior
VAL-03	Alphabet entry with multiple characters	Entry is rejected
VAL-04	Undefined start state	Entry is rejected
VAL-05	Undefined accepting state	Entry is rejected
VAL-06	Undefined transition destination	Entry is rejected
VAL-07	Numeric entry outside allowed range	Entry is rejected

Test Environment

This was tested in C++ on GitHub g++. A windows computer was used with main.cpp being the source file from which the simulator was compiled.

Test Results

DFA Results

Test ID	Actual Result	Status
DFA-01	Input 1 ended in accepting state q1.	Pass
DFA-02	Input 101 ended in accepting state q1.	Pass
DFA-03	Input 100 ended in non-accepting state q0.	Pass

Test ID	Actual Result	Status
DFA-04	EMPTY remained in non-accepting start state q0.	Pass
DFA-05	Input 102 reported invalid symbol 2.	Pass
DFA-06	QUIT closed the simulator normally.	Pass

NFA Results

Test ID	Actual Result	Status
NFA-01	Input 000 left only non-accepting state q0 active.	Pass
NFA-02	Input 001 produced active states q0 and q1 and was accepted.	Pass
NFA-03	Input 1010 retained accepting state q1 and was accepted.	Pass
NFA-04	EMPTY remained in non-accepting start state q0.	Pass
NFA-05	Input 12 reported invalid symbol 2.	Pass
NFA-06	QUIT closed the simulator normally.	Pass

Validation Results

Test ID	Actual Result	Status
VAL-01	Invalid automata type BAD was rejected.	Pass
VAL-02	Duplicate state q0 was rejected.	Pass
VAL-03	Multi-character symbol 00 was rejected.	Pass
VAL-04	Undefined start state bad was rejected.	Pass
VAL-05	Undefined accepting state bad was rejected.	Pass
VAL-06	Undefined transition destination bad was rejected.	Pass
VAL-07	Out-of-range numeric entries were rejected.	Pass

Test Summary

All 19 planned DFA, NFA, and validation test cases passed. The program compiled successfully using g++ with the C++17 standard in GitHub Codespaces. It displayed the expected state-transition sequences, acceptance results, input validation messages, and normal termination behavior. No functional defects remain.